

Corey Zhou

coreyzh@gmail.com | (510) 329-7168 | linkedin.com/in/corey-z-2a7781231 | San Leandro, CA

OBJECTIVE

Computer engineering graduate seeking a Design Verification Engineer role. Hands-on experience verifying a RISC-V processor using constrained-random testing, SystemVerilog assertions, with self-directed study in UVM. Fast learner with strong programming and debugging fundamentals and a track record of taking ownership of complex technical problems.

TECHNICAL SKILLS

Hardware Description & Verification: SystemVerilog, UVM (coursework/self-study), Verilog, Constrained-Random Verification, SystemVerilog Assertions (SVA), Functional Coverage, MIPS32 Assembly

EDA / Simulation Tools: ModelSim, HSPICE, Vivado

Programming Languages: C/C++, Python, Java, JavaScript/TypeScript

Scripting & Test Automation: Pytest, Selenium, Jest

Systems: Linux/Unix, Windows

Data & Machine Learning: TensorFlow, Keras, Scikit-learn, NumPy, Pandas, Matplotlib

Additional: Docker, Jenkins, Git, CI/CD, AWS, Terraform, Networking (TCP/IP, ARP)

EDUCATION

University of California, Santa Barbara — *B.S. Computer Engineering*

GPA: 3.78 | Dean's Honors

Relevant Self-Study (Udemy): SystemVerilog Verification Series; CI/CD with Jenkins & Docker; Terraform for AWS;

HARDWARE DESIGN PROJECTS

Dual-Port RAM UVM Verification Environment

- Built a complete UVM verification environment (driver, monitor, agent, scoreboard, sequence, subscriber) to verify a synchronous dual-port RAM with independent read and write ports
- Wrote a sequence generating randomized, pipelined transactions that write to a random address and read from the previously written address in the same cycle, exercising concurrent read/write behavior
- Built a scoreboard with a reference memory model to compare DUT output against expected data on every read, flagging mismatches as errors

5-Stage 32-bit Pipelined MIPS32 Processor

- Designed and verified a complete 5-stage pipelined MIPS32 processor (fetch, decode, execute, memory, writeback) in Verilog, with a hazard unit for data forwarding and stall detection
- Built a 2-bit, 2-way branch predictor, a 32KB 2-way set-associative cache (4-word blocks), and a combinational multiplier for multiply/divide with a dedicated testbench per module

Full-Custom 16-bit Carry Skip Adder

- Designed a full-custom 16-bit carry skip adder using pass transistor logic, sense amplifiers, and complementary domino logic to generate PGK sum signals at full voltage swing
- Implemented carry-skip propagation across adder blocks to reduce critical path delay
- Simulated every custom cell and the full adder in HSpice, characterizing delay and power across PVT corners to validate design margins

RISC-V Processor Functional Verification

- Built a SystemVerilog verification environment with constrained-random assembly test programs and SVA checks to detect functional bugs in a RISC-V processor pipeline
- Debugged simulation failures and traced root causes through the pipeline to verify correct behavior

EXPERIENCE

SlamOptix — Founding Engineer

- Joined as founding engineer with full ownership of architecture and technical direction, building core products end-to-end as the sole engineer (MongoDB, Node.js, Next.js, NGINX)
- Built an internal AI-assisted development pipeline (self-hosted LLM inference + automated test/integration) that reduced manual effort on routine engineering tasks

Independent Contractor — Test Automation & Security

- Built automated test suites (Python, Pytest, Selenium) to validate web application functionality end-to-end; designed a full-stack device-monitoring app (Node.js, Next.js, MariaDB, Docker Compose)
- Performed protocol-level security testing (Kali Linux, Scapy, NMap, Wireshark) to identify DHCP and network-layer vulnerabilities, applying a systematic, spec-driven testing methodology directly transferable to hardware verification

CK-12 Foundation — Math Interactives Intern: Palo Alto, CA

- Designed interactive middle/high school math content and performed QA review on other interns' work using GeoGebra's JavaScript API